

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

Semester: Fall 2026-2027

BACSE202 - Database Management Systems

Faculty Name: Dr. S. Geetha

CARE CONTINUUM

A TELEMEDICINE CONSULTATION AND ELECTRONIC HEALTH RECORDS SYSTEM

Team Members

Dhuna Chavda [25BCE1072]

Pratyush Deo Singh [25BCE5101]

Sarbajeet Pattadar [25BCE1920]

Links

GitHub Repository Link

GitHub Repository

Demo Video Link

Demo Video

Table of Contents

1. Abstract
2. Keywords
3. Introduction
4. Existing Systems
5. Proposed system architecture
6. Relational database system
7. Relational mapping of ER Diagram
8. Functional dependencies
9. Normalization
10. Cassandrea implementation
11. Implementation
12. Features
13. Testing and results
14. Limitations
15. Future enhancements
16. Conclusion
17. References

1 Abstract

Care Continuum is a telemedicine and Electronic Health Record (EHR) management system developed to store, manage, and track patient healthcare information. It supports registration of patients, booking of appointments with doctors for both in-person and virtual consultations , prescription of medicines, ordering of lab tests, billings, and retrieval of medical history of each patient. The medical history includes appointments, prescriptions, allergies notes, and laboratory tests which are then displayed in a chronological order for convenient access.

In comparison to the traditional database systems, our project provides access to all of this information at a single point of access, thus, reducing data loss, avoiding duplication and reducing operational delays. Built on a normalised Oracle SQL relational database, it guarantees atomicity, consistency, isolation and durability. Furthermore, this system follows a hybrid model of amalgamation of Oracle SQL relational database and Apache Cassandra NoSQL database to handle scalability problems such as accessing patient health records.

2 Keywords

Electronic Health Records (EHR), Telemedicine, Database Management System (DBMS), Oracle Autonomous Database, Apache Cassandra, Hybrid Database Architecture, Relational Database, NoSQL, PL/SQL, Patient Medical Timeline.

3 Introduction

BACKGROUND

The healthcare sector generates and manages large amounts of information. Efficient management of this information is essential to ensure continuity and smooth operation for healthcare professionals to access relevant information about the patients when required.

In many healthcare environments, the necessary information is scattered across different record keeping systems which makes data retrieval difficult and time consuming. This fragmentation also results in duplication and inconsistency.

Care Continuum is a telemedicine and EHR management system developed to address this challenge through a centralized system. It provides functionalities such as patient registration, appointment scheduling, prescription of medicines, and lab tests and their results. The system aims to simplify the workflow of healthcare information.

The project uses Oracle Autonomous Database as its primary relational database. A normalized relational schema is implemented using primary keys, foreign keys, constraints, and various relationships. PL/SQL procedures and triggers are used to implement rules and automate certain operations.

To support chronological retrieval of a patient's medical history, the system also has Apache Cassandra, a NoSQL database. Selected clinical records from Oracle are loaded into Cassandra and organized according to patient ID and month and then displayed in the latest-first fashion.

PROBLEM STATEMENT

The existing healthcare system uses fragmented databases, where data about patients, doctors, laboratories and pharmacies is stored in separate databases. This might lead to increased data retrieval costs and even data duplication or loss. Which also results in difficulty managing huge volumes of logs in case the history of a patient grows too much.

OBJECTIVES

1. Design a relational database schema to store information regarding patients, doctors, appointments, prescriptions, lab order and so on
2. Ensure normalisation of relational tables up to BCNF
3. Implement relational database using Oracle SQL to store operational medical records
4. Implement modern database using Apache Cassandra to store patient medical health records

SCOPE

In Scope

1. A website that allows access to electronic health records and appointment details
2. Registering new patients, booking appointments, requesting lab orders, prescribing medicines in accordance to the medical history and updations for the same.
3. A medical timeline of the patient which is organised according to patient's ID and the month it was created in and sorted by date (latest first)

Out of Scope

1. A mobile application is out of scope for this project
2. If a virtual appointment is booked, the website does not support direct video calling. Instead, a meeting URL can be stored in the database which can be accessed.
3. Documents are not directly stored in the database. The links to their location in a cloud storage (like Google Drive) are allowed to be stored instead.

4 Existing systems

TELEMEDICINE PLATFORMS

1. eSanjeevani
2. Apollo TeleHealth
3. Practo
4. Tata 1mg

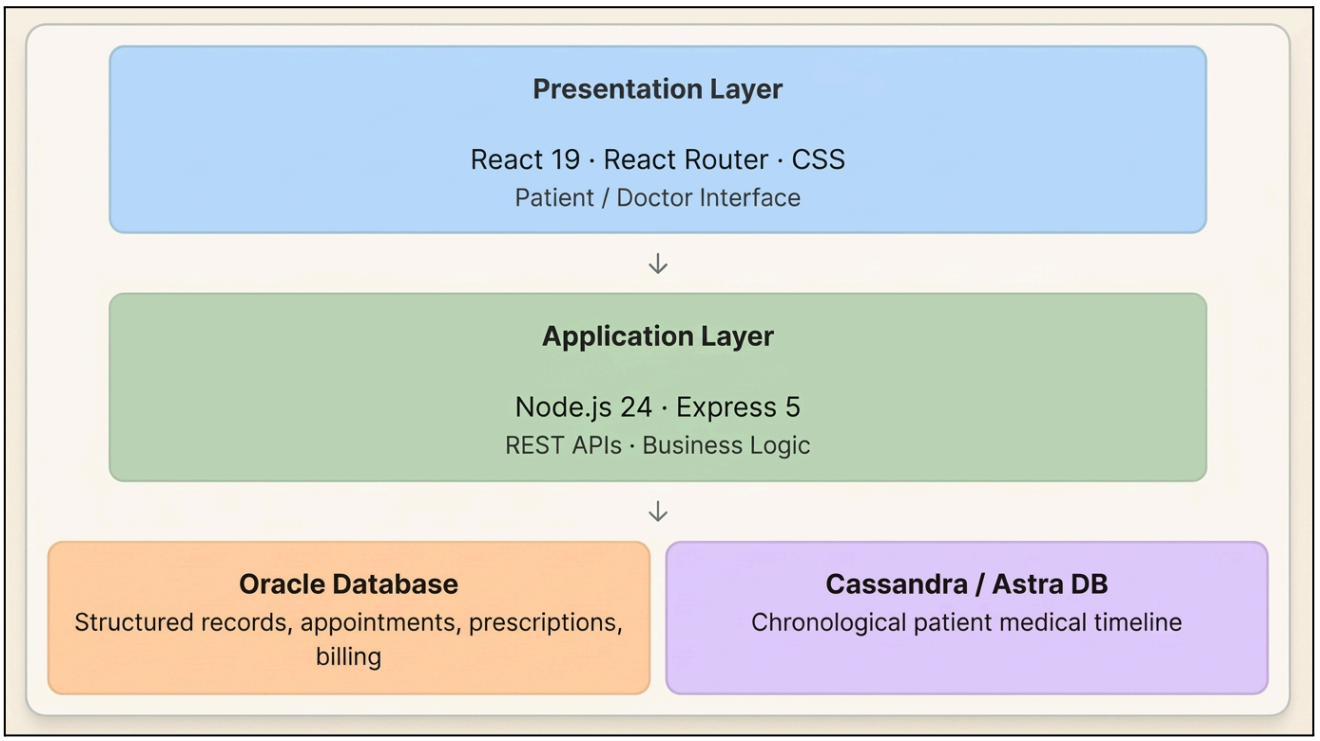
ELECTRONIC HEALTH RECORDS SYSTEMS

1. PrognoCis
2. Epic EHR
3. Azalea Health

TELEMEDICINE AND EHR PLATFORMS

1. DrChrono
2. Kareo
3. athenaOne EHR

5 Proposed System Architecture



The user interacts with the React frontend. The frontend communicates with the Node.js and Express REST API. The backend has the application logic. It then communicates with both the databases, Oracle and Cassandra. Oracle handles the relational operational data and Cassandra is used for chronological patient timeline.

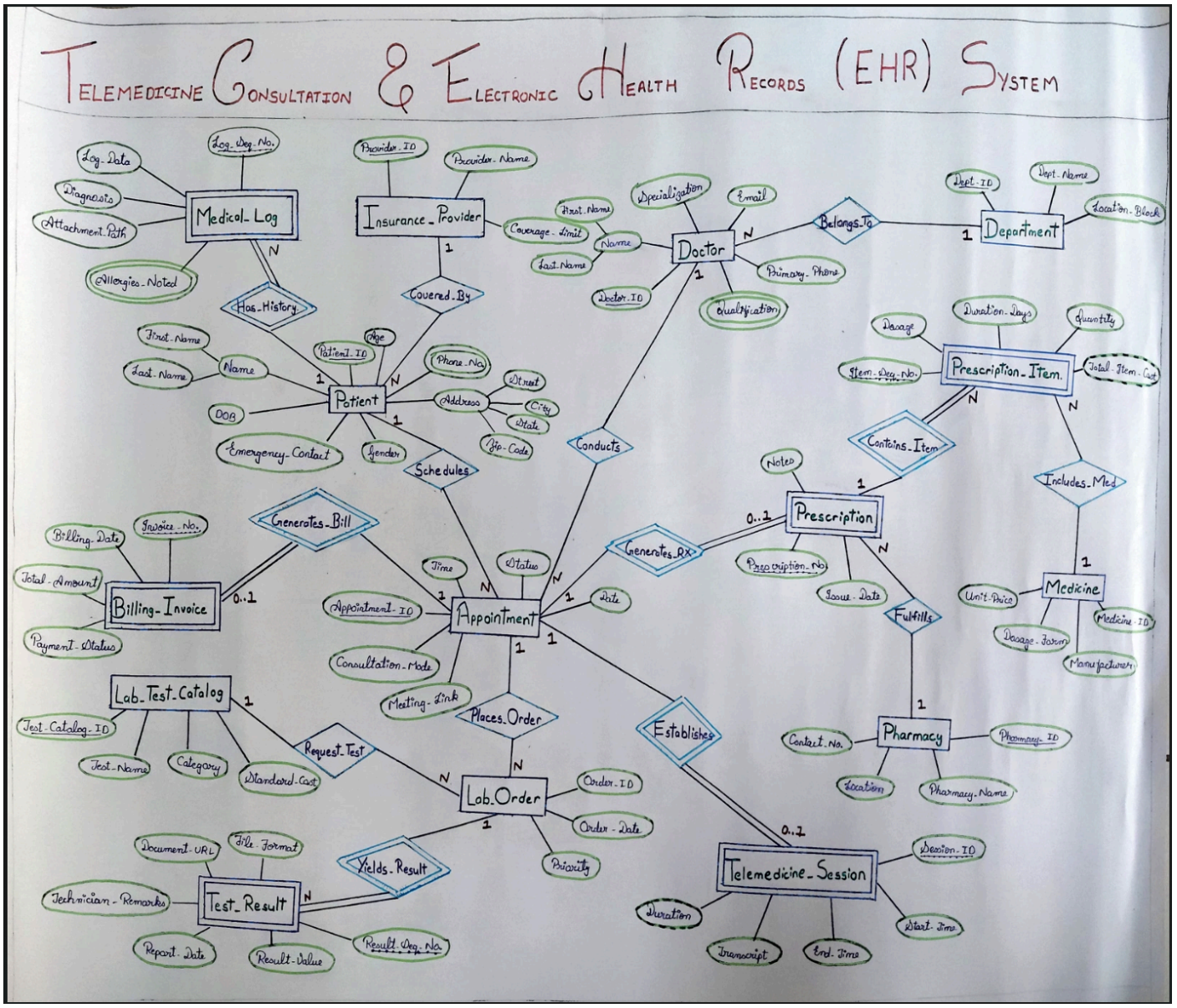
Method	Path	Operation
GET	`/session`	Check login state
POST	`/login`	Sign in with the demo password
POST	`/logout`	Clear the session cookie
GET	`/test-db`	Check Oracle connectivity
GET	`/patients`	List patients
GET	`/doctors`	List doctors and departments
GET	`/medicines`	List medicines, prices, and allergen classes
GET	`/pharmacies`	List pharmacies
GET	`/medical-logs`	List patient medical history
GET	`/allergies`	List allergies by patient and log entry

GET	`/appointments`	List joined appointment details
GET	`/prescriptions`	List prescription medicine items
GET	`/lab-catalog`	List available lab tests and costs
GET	`/lab-orders`	List lab orders
GET	`/test-results`	List reported results
GET	`/telemedicine-sessions`	List virtual-session metadata
GET	`/timeline/health`	Check timeline configuration and Cassandra access
POST	`/timeline/import`	Refresh all patients from a read-only Oracle snapshot; 200 complete, 207 partial, 409 instance busy, 413 dataset limit, 408 source-read budget, 503 unavailable
GET	`/timeline/:patient_id?month=YYYY-MM`	Read a Cassandra partition; optional `limit` and signed `cursor`
GET	`/billing`	List stored and calculated invoice totals
GET	`/billing/:appointment_id/items`	List current medicine and lab charges
POST	`/patients`	Register a patient through PL/SQL
POST	`/appointments`	Book an appointment
POST	`/prescriptions`	Create a prescription and its items together
POST	`/medical-logs`	Record history and allergies together
POST	`/lab-orders`	Order a catalog test
POST	`/test-results`	Record a result for an order
POST	`/billing`	Generate a calculated pending invoice
PUT	`/appointments/:appointment_id/status`	Update status through PL/SQL
PUT	`/billing/:appointment_id/:invoice_no`	Recalculate, pay, or refund an invoice

6 Relational Database Design

ER DIAGRAM

[Link to ER Diagram]



STRONG ENTITIES

NAME	PRIMARY KEY
PATIENT	Patient_ID
INSURANCE_PROVIDER	Policy_No
DOCTOR	Doctor_ID
DEPARTMENT	Dept_ID
APPOINTMENT	Appointment_ID
MEDICINE	Medicine_ID
PHARMACY	Pharmacy_ID
LAB_ORDER	Order_ID
LAB_TEST_CATALOG	Test_Catalog_ID

WEAK ENTITIES

NAME	PRIMARY KEY
MEDICAL_LOG	Patient_ID + Log_Seq_No
TELEMEDICINE SESSION	Appointment_ID + Session_ID
PRESCRIPTION	Appointment_ID + Prescription_No
PRESCRIPTION_ITEM	Appointment_ID + Prescription_No + Item_Seq_No
TEST_RESULT	Order_ID + Result_Seq_No
BILLING_INVOICE	Appointment_ID + Invoice_No

RELATIONSHIPS

NAME	BETWEEN	Cardinality
COVERED_BY	PATIENT + INSURANCE_PROVIDER	N:1
HAS_HISTORY	PATIENT + MEDICAL_LOG	1:N
BELONGS_TO	DOCTOR + DEPARTMENT	N:1
SCHEDULES	PATIENT + APPOINTMENT	1:N
CONDUCTS	DOCTOR + APPOINTMENT	1:N
ESTABLISHES	APPOINTMENT + TELEMEDICINE_SESSION	1:0..1
GENERATES_RX	APPOINTMENT + PRESCRIPTION	1:0..1
CONTAINS_ITEM	PRESCRIPTION + PRESCRIPTION_ITEM	1:N
INCLUDES_MED	PRESCRIPTION_ITEM + MEDICINE	N:1
FULFILLS	PRESCRIPTION + PHARMACY	N:1
PLACES_ORDER	APPOINTMENT + LAB_ORDER	1:N
REQUEST_TEST	LAB_ORDER + LAB_TEST_CATALOG	N:1
YIELDS_RESULT	LAB_ORDER + TEST_RESULT	1:N
GENERATES_BILL	APPOINTMENT + BILLING_INVOICE	1:0..1

7 Relational Mapping of the ER Model

1. PATIENT (Patient_ID, First_Name, Last_Name, Street, City, State, Zip_Code, DOB, Gender, Emergency_Contact, Policy_No)

   FK: Policy_No → INSURANCE_PROVIDER (Policy_No)

2. PATIENT_PHONE (Patient_ID, Phone_Number)

   FK: Patient_ID → PATIENT (Patient_ID)

3. DOCTOR (Doctor_ID, First_Name, Last_Name, Specialization, Email, Primary_Phone, Dept_ID)

   FK: Dept_ID → DEPARTMENT (Dept_ID)

4. DEPARTMENT (Dept_ID, Dept_Name, Location_Block)

5. DOCTOR_QUALIFICATION (Doctor_ID, Qualification)

   FK: Doctor_ID → DOCTOR (Doctor_ID)

6. APPOINTMENT (Appointment_ID, Patient_ID, Doctor_ID, Appointment_Date, Appointment_Time, Status, Consultation_Mode, Meeting_Link)

   FK: Patient_ID → PATIENT (Patient_ID)
   Doctor_ID → DOCTOR (Doctor_ID)

7. PHARMACY (Pharmacy_ID, Pharmacy_Name, Location, Contact_Number)

8. MEDICINE (Medicine_ID, Medicine_Name, Manufacturer, Allergen_Class, Dosage_Form, Unit_Price)

9. LAB_TEST_CATALOG (Test_Catalog_ID, Test_Name, Category, Standard_Cost)

10. INSURANCE_PROVIDER (Policy_No, Provider_Name, Coverage_Limit)

11. PRESCRIPTION (Appointment_ID, Prescription_No, Issue_Date, Notes, Pharmacy_ID)

    FK: Appointment_ID → APPOINTMENT (Appointment_ID)
    Pharmacy_ID → PHARMACY (Pharmacy_ID)

12. PRESCRIPTION_ITEM (Appointment_ID, Prescription_No, Item_Seq_No, Medicine_ID, Dosage, Duration_Days, Quantity)

    FK: (Appointment_ID, Prescription_No) → PRESCRIPTION (Appointment_ID,

Prescription_No)
Medicine_ID → MEDICINE (Medicine_ID)

13. LAB_ORDER (Order_ID, Appointment_ID, Test_Catalog_ID, Order_Date, Priority)

    FK: Appointment_ID → APPOINTMENT (Appointment_ID)
    Test_Catalog_ID → LAB_TEST_CATALOG (Test_Catalog_ID)

14. TEST_RESULT (Order_ID, Result_Seq_No, Result_Value, Report_Date, Technician_Remarks, Document_URL, File_Format)

    FK: Order_ID → LAB_ORDER (Order_ID)

15. MEDICAL_LOG (Patient_ID, Log_Seq_No, Log_Date, Diagnosis, Attachment_Path)

    FK: Patient_ID → PATIENT (Patient_ID)

16. LOG_ALLERGIES (Patient_ID, Log_Seq_No, Allergy_Name)

    FK: (Patient_ID, Log_Seq_No) → MEDICAL_LOG (Patient_ID, Log_Seq_No)

17. BILLING_INVOICE (Appointment_ID, Invoice_No, Billing_Date, Total_Amount, Payment_Status)

    FK: Appointment_ID → APPOINTMENT (Appointment_ID)

18. TELEMEDICINE_SESSION (Appointment_ID, Session_ID, Start_Time, End_Time, Network_Quality_Log, Chat_Transcript_URL)

    FK: Appointment_ID → APPOINTMENT (Appointment_ID)

8 Functional Dependencies

PATIENT:
Patient_ID → First_Name, Last_Name, Street, City, State, Zip_Code, DOB, Gender, Emergency_Contact, Policy_No

DOCTOR:
Doctor_ID → First_Name, Last_Name, Specialization, Email, Primary_Phone, Dept_ID

DEPARTMENT:
Dept_ID → Dept_Name, Location_Block

APPOINTMENT:
Appointment_ID → Patient_ID, Doctor_ID, Appointment_Date, Appointment_Time, Status, Consultation_Mode, Meeting_Link

PHARMACY:
Pharmacy_ID → Pharmacy_Name, Location, Contact_Number

MEDICINE:
Medicine_ID → Medicine_Name, Manufacturer, Dosage_Form, Unit_Price, Allergen_Class

LAB_TEST_CATALOG:
Test_Catalog_ID → Test_Name, Category, Standard_Cost

INSURANCE_PROVIDER:
Policy_No → Provider_Name, Coverage_Limit

PRESCRIPTION:
(Appointment_ID, Prescription_No) → Issue_Date, Notes, Pharmacy_ID

PRESCRIPTION_ITEM:
(Appointment_ID, Prescription_No, Item_Seq_No) → Medicine_ID, Dosage, Duration_Days, Quantity

LAB_ORDER:
Order_ID → Appointment_ID, Test_Catalog_ID, Order_Date, Priority

TEST_RESULT:
(Order_ID, Result_Seq_No) → Result_Value, Report_Date, Technician_Remarks, Document_URL, File_Format

MEDICAL_LOG:
(Patient_ID, Log_Seq_No) → Log_Date, Diagnosis, Attachment_Path

BILLING_INVOICE:
(Appointment_ID, Invoice_No) → Billing_Date, Total_Amount, Payment_Status

TELEMEDICINE_SESSION:
(Appointment_ID, Session_ID) → Start_Time, End_Time, Network_Quality_Log, Chat_Transcript_URL

9 Normalisation

1NF

Requirements:

- ➢ Attributes must contain atomic values.
- ➢ No repeating groups.
- ➢ No multivalued attributes.

1NF Transformations:

1. PATIENT Decomposition:
   Name -> First_Name, Last_Name
   Address -> Street, City, State, Zip_Code .

   PATIENT (Patient_ID, First_Name, Last_Name, Street, City, State, Zip_Code, DOB, Gender, Emergency_Contact, Policy_No)

   Functional Dependency:
   Patient_ID -> First_Name, Last_Name, Street, City, State, Zip_Code, DOB, Gender, Emergency_Contact, Policy_No

2. DOCTOR Decomposition:
   Name -> First_Name, Last_Name

   DOCTOR (Doctor_ID, First_Name, Last_Name, Specialization, Email, Primary_Phone, Dept_ID)

   Functional Dependency:
   Doctor_ID -> First_Name, Last_Name, Specialization, Email, Primary_Phone, Dept_ID

Multivalued Attributes:

1. PATIENT (Phone_Number) Created: PATIENT_PHONE (Patient_ID, Phone_Number)
   FK: Patient_ID -> PATIENT (Patient_ID)

2. DOCTOR (Qualification) Created: DOCTOR_QUALIFICATION (Doctor_ID, Qualification)
   FK: Doctor_ID -> DOCTOR (Doctor_ID)

Derived Attributes not stored:

Age: Calculated from DOB
Total_Item_Cost: Quantity x Unit_Price
Duration_Minutes: End_Time - Start_Time

2NF

Requirements:

- ➢ Relation must be in 1NF.
- ➢ There must be no partial dependency.
- ➢ Every non-prime attribute must depend on the whole candidate key.
- ➢ If the candidate key has only one attribute, partial is not possible.

Analysis:

- ➢ PATIENT_PHONE, DOCTOR_QUALIFICATION, LOG_ALLERGIES:
  No non-trivial FD. Hence, no Partial Dependency.

- ➢ PRESCRIPTION, PRESCRIPTION_ITEM, TEST_RESULT, MEDICAL_LOG, BILLING_INVOICE, TELEMEDICINE_SESSION:
  For these relations, the non-prime attributes depend on the complete candidate key as shown in the list of Functional Dependencies.

Other relations have single-attribute primary keys, so partial dependency is not possible. Hence, no decomposition required. All 18 relations are in 2NF.

3NF

Requirements:

- ➢ Relation must be in 2NF.
- ➢ There must be no transitive dependency of a non-prime attribute on a candidate key.
- ➢ For every non-trivial FD X -> Y: X must be a superkey OR Y must be a prime attribute.

Analysis:

In the present schema, for every non-trivial FD listed in Functional Dependencies, the determinant is a superkey. No transitive dependency among non-key attributes is given. Thus, all 18 relations are already in 3NF. No decomposition required.

BCNF

Requirements:

- ➢ Relation must be in 3NF.
- ➢ For every non-trivial FD X -> Y, X must be a superkey.
- ➢ BCNF is stricter than 3NF because there is no alternative condition involving prime attributes.

Analysis:

As verified in 3NF, every non-trivial FD in the schema has a superkey (primary key) as its determinant. Thus, all 18 relations are in BCNF. No decomposition required.

10 Cassandra Implementation

CASSANDRA TABLE DESIGN

Table: clinical_events_by_patient_month
Purpose: to store a chronological timeline of a patient's clinical events

COLUMN	PURPOSE
patient_id	Identifies the patient
event_month	Month in which the event occured
event_time	Date/time used to order the records
event_id	Unique identifier for the timeline event
event_type	Type of event - appointment, prescription, lab result, etc.
patient_name	Patient name included for timeline display
appointment_id	Associated appointment, where applicable
summary	A short description of the clinical events
source_record_id	Identifier of the original Oracle record

Partition key: (patient_id, event_month)

Clustering columns: event_time (ordered DESC), event_id

TRANSFER OF ORACLE RECORDS TO CASSANDRA

The backend has a Refresh from Oracle option for updating the Cassandra timeline. When we click refresh, the backend opens a read-only Oracle transaction and retrieves the data. It then converts this data into events containing the necessary fields and then inserted into Cassandra using prepared CQL statements.

The transfer of data is not atomic, i.e., if some of the data is not transferred to the Cassandra DB, the transferred data stays on the Cassandra DB, and if the other data is needed, a refresh can be performed to load them.

The refresh from Oracle Autonomous Database to Apache Cassandra is not automatic. It has to be performed manually from the website.

11 Implementation

DATABASE MANAGEMENT SYSTEM

Care Continuum implements a hybrid database architecture using Oracle Autonomous Database and Apache Cassandra.

Oracle Autonomous Database:
It is used as the primary relational database for storing and managing operational data stored in 18 tables that cover patient registration, doctor information, appointment booking, medical records, prescriptions, lab tests, and billing.

Apache Cassandra:
It is used as the NoSQL component of the system. Its main purpose is to store and manage massive volumes of data. Oracle remains the source of the original records and handles almost everything. Cassandra on the other hand, is used to view a whole timeline of the events related to the medical workflow - from booking of the appointment, to the prescription of medicines including lab orders and their results.

APPLICATION TECHNOLOGIES

Frontend
React 19, React Router 7, Lucide icons, CSS, Vite 8, Oxlint

Backend and API
Node.js 24, Express 5

Relational Database
Oracle, SQL/PLSQL, node-oracledb 7

Modern Database
Apache Cassandra 5.0.9 locally; Astra DB Free hosted; cassandra-driver 4.10

Hosting
Vercel, Oracle Autonomous Database

Others
Docker Compose (for Cassandra process, health checks, and persistent storage)

DATABASE CONNECTIVITY

Oracle Database Connectivity

The Node.js backend connects to Oracle Autonomous Database using the node-oracledb driver.

Cassandra Database Connectivity

The backend connects to Apache Cassandra using the Cassandra driver for Node.js

DATA FLOW AND INTEGRATION

1. The user interacts with the React frontend
2. The frontend sends a HTTP request to the backend
3. The backend process the request
4. The backend communicated with Oracle or Cassandra, depending on the operation
5. Oracle handles operational health records while Cassandra handles retrieval of patient timeline
6. The backend returns the requested data to the frontend via API

The architecture separates the user interface, application logic, and database operations.

12 Features

12.1 Storing and retrieval of operational medical records

The system stores data related to patients, doctors, appointments, prescriptions, lab orders, etc. and presents them in an organised way to the user.

```
CREATE TABLE PATIENT (
    Patient_ID VARCHAR2(12),
    Policy_No VARCHAR2(20),
    First_Name VARCHAR2(35) NOT NULL,
    Last_Name VARCHAR2(35) NOT NULL,
    Street VARCHAR2(80),
    City VARCHAR2(35),
    State VARCHAR2(35),
    Zip_Code VARCHAR2(10),
    DOB DATE NOT NULL,
    Gender VARCHAR2(10),
    Emergency_Contact VARCHAR2(15),
    CONSTRAINT pk_patient PRIMARY KEY (Patient_ID),
    CONSTRAINT fk_patient_insurance FOREIGN KEY (Policy_No) REFERENCES INSURANCE_PROVIDER(Policy_No) ON DELETE SET NULL
);

CREATE TABLE DOCTOR (
    Doctor_ID VARCHAR2(10),
    Dept_ID VARCHAR2(10) NOT NULL,
    First_Name VARCHAR2(35) NOT NULL,
    Last_Name VARCHAR2(35) NOT NULL,
    Specialization VARCHAR2(60) NOT NULL,
    Email VARCHAR2(80) UNIQUE NOT NULL,
    Primary_Phone VARCHAR2(15) NOT NULL,
    CONSTRAINT pk_doctor PRIMARY KEY (Doctor_ID),
    CONSTRAINT fk_doctor_dept FOREIGN KEY (Dept_ID) REFERENCES DEPARTMENT(Dept_ID)
);
```

12.2 Storing and retrieval of chronological medical timeline of patients

The system loads data from Oracle DB to Cassandra DB, where it gets organised in terms of patient ID and event month, which then can be accessed via the website.

```
CREATE TABLE IF NOT EXISTS care_continuum.clinical_events_by_patient_month (
    patient_id text,
    event_month text,
    event_time timestamp,
    event_id text,
    event_type text,
    patient_name text,
    appointment_id text,
    summary text,
    source_record_id text,
    PRIMARY KEY ((patient_id, event_month), event_time, event_id)
) WITH CLUSTERING ORDER BY (event_time DESC, event_id ASC);
```

Creation of the timeline table

```
INSERT INTO care_continuum.clinical_events_by_patient_month
(patient_id, event_month, event_time, event_id, event_type, summary)
VALUES ('DEMO-CRUD', '2026-09', '2026-09-01T00:00:00Z',
        'crud:1', 'DEMONSTRATION', 'Initial demonstration summary');
```

Insertion of a demonstration record

```
SELECT event_time, event_id, event_type, summary
FROM care_continuum.clinical_events_by_patient_month
WHERE patient_id = 'DEMO-01' AND event_month = '2026-09';
```

Retrieving a patient's record for 1 month

12.3 Function to calculate bill

A PL/SQL function has been defined which calculates the billing amount related to an appointment by summing up the amount for consultation, prescription and lab test.

```
CREATE OR REPLACE FUNCTION Calculate_Invoice_Total (
    p_appointment_id IN VARCHAR2
) RETURN NUMBER
IS
    v_medicine_total NUMBER(10, 2) := 0;
    v_lab_total NUMBER(10, 2) := 0;
BEGIN
    SELECT NVL(SUM(pi.Quantity * m.Unit_Price), 0)
    INTO v_medicine_total
    FROM PRESCRIPTION_ITEM pi
    JOIN MEDICINE m ON m.Medicine_ID = pi.Medicine_ID
    WHERE pi.Appointment_ID = p_appointment_id;

    SELECT NVL(SUM(lt.Standard_Cost), 0)
    INTO v_lab_total
    FROM LAB_ORDER lo
    JOIN LAB_TEST_CATALOG lt ON lt.Test_Catalog_ID = lo.Test_Catalog_ID
    WHERE lo.Appointment_ID = p_appointment_id;

    RETURN v_medicine_total + v_lab_total;
END;
/
```

12.4 Trigger to create telemedicine session

When the ‘virtual’ option is selected in consultation mode while booking an appointment, it triggers the creation of a record in the TELEMEDICINE_SESSION in order to store information for it.

```
CREATE OR REPLACE TRIGGER trg_create_telemedicine_session
AFTER INSERT ON APPOINTMENT
FOR EACH ROW
WHEN (NEW.Consultation_Mode = 'Virtual')
BEGIN
    INSERT INTO TELEMEDICINE_SESSION (
        Appointment_ID,
        Session_ID,
        Start_Time,
        End_Time,
        Network_Quality_Log,
        Chat_Transcript_URL
    )
    VALUES (
        :NEW.Appointment_ID,
        'SES-' || SUBSTR(:NEW.Appointment_ID, 5),
        NULL,
        NULL,
        'Session created automatically',
        NULL
    );
END;
/
```

12.5 Trigger to avoid allergic prescription

If a patient has his/her allergic class stored in the LOG_ALLERGIES tables, while the doctor is prescribing a medicine to the patient for a particular appointment, the allergies will be displayed on the top when the patient is selected from the list. In case the doctor selects a medicine (allergen class of the medicine stored in the MEDICINE table) which the patient is allergic to, it will not allow the prescription to be created at all.

```
CREATE OR REPLACE TRIGGER Prevent_Allergic_Prescription
BEFORE INSERT OR UPDATE OF Medicine_ID ON PRESCRIPTION_ITEM
FOR EACH ROW
DECLARE
    v_patient_id PATIENT.Patient_ID%TYPE;
    v_medicine_name MEDICINE.Medicine_Name%TYPE;
    v_allergen_class MEDICINE.Allergen_Class%TYPE;
    v_conflict_count NUMBER;
BEGIN
    SELECT a.Patient_ID
    INTO v_patient_id
    FROM APPOINTMENT a
    WHERE a.Appointment_ID = :NEW.Appointment_ID;

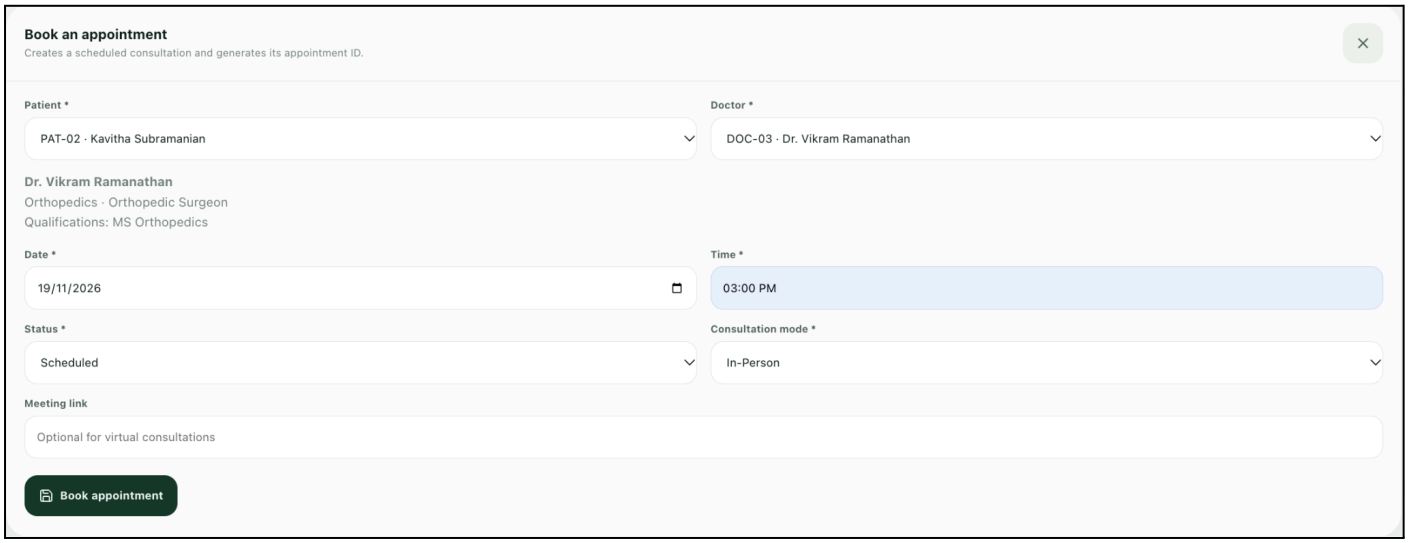
    SELECT m.Medicine_Name, m.Allergen_Class
    INTO v_medicine_name, v_allergen_class
    FROM MEDICINE m
    WHERE m.Medicine_ID = :NEW.Medicine_ID;

    IF v_allergen_class IS NOT NULL THEN
        SELECT COUNT(*)
        INTO v_conflict_count
        FROM LOG_ALLERGIES la
        WHERE la.Patient_ID = v_patient_id
          AND UPPER(TRIM(la.Allergy_Name)) = UPPER(TRIM(v_allergen_class));

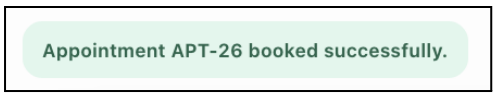
        IF v_conflict_count > 0 THEN
            RAISE_APPLICATION_ERROR(
                -20002,
                'Allergy conflict: ' || v_medicine_name ||
                ' belongs to allergen class ' || v_allergen_class ||
                ' for patient ' || v_patient_id
            );
        END IF;
    END IF;
END;
/
```

13 Testing and Results

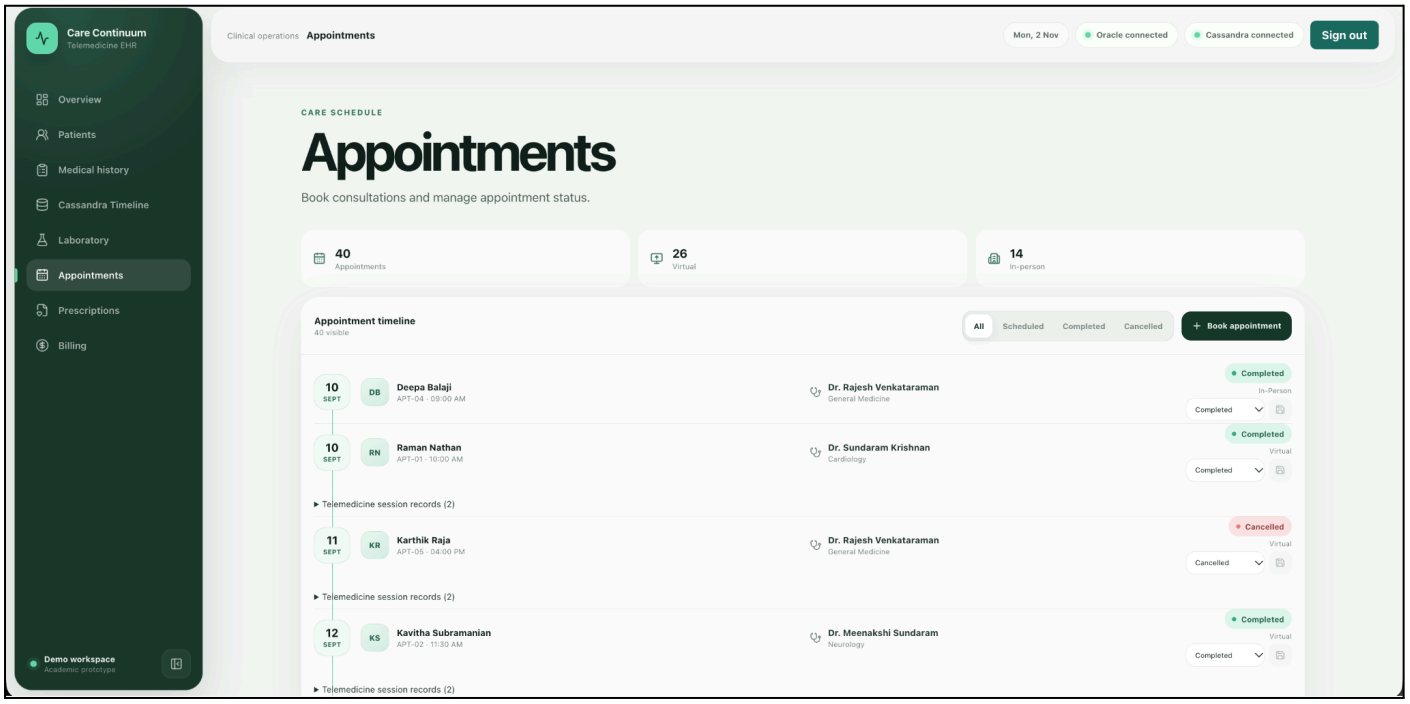
TEST: Booking an appointment (Creation)



RESULT: Appointment booked successfully

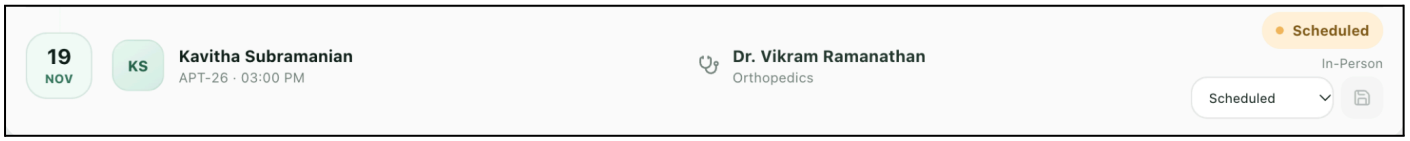


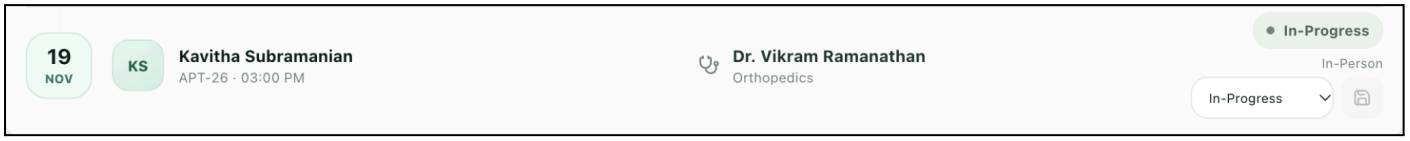
TEST: Displaying appointments (Reading)

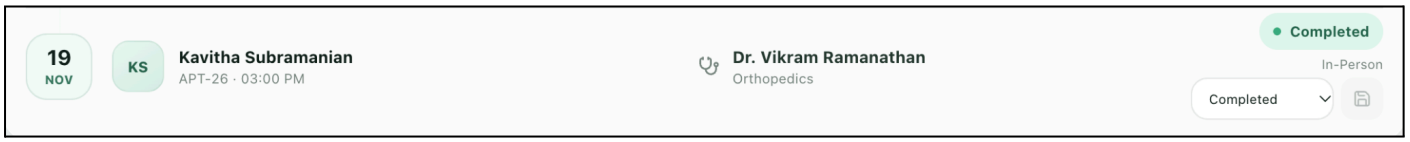


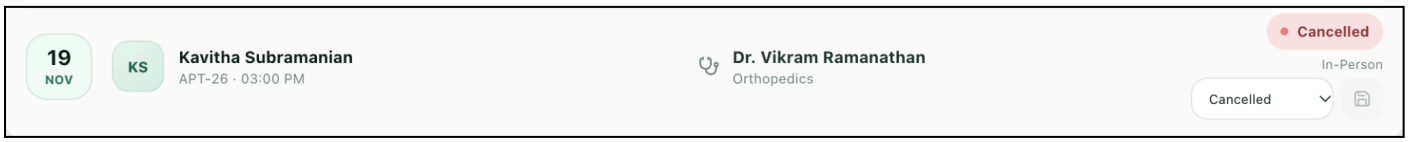
RESULT: All booked appointments are visible

TEST: Changing appointment status (Updation)









RESULT: Updating appointment status works successfully

TEST: Deleting a record (Deletion)

Note: Deletion is implemented database level only and not website level

Before deletion

```
cqlsh:care_continuum> INSERT INTO clinical_events_by_patient_month
                  ... (patient_id, event_month, event_time, event_id, event_type, summary)
                  ... VALUES (
                  ...     'DEMO-CRUD',
                  ...     '2026-09',
                  ...     '2026-09-01T00:00:00Z',
                  ...     'crud:1',
                  ...     'DEMONSTRATION',
                  ...     'Temporary record for DELETE testing'
                  ... );
cqlsh:care_continuum> SELECT patient_id, event_month, event_time, event_id, event_type, summary
                  ... FROM clinical_events_by_patient_month
                  ... WHERE patient_id = 'DEMO-CRUD'
                  ... AND event_month = '2026-09';

 patient_id | event_month | event_time                      | event_id | event_type    | summary
------------+-------------+---------------------------------+----------+---------------+-------------
------------------------
   DEMO-CRUD |     2026-09 | 2026-09-01 00:00:00.000000+0000 |   crud:1 | DEMONSTRATION | Temporary rec
ord for DELETE testing

(1 rows)
cqlsh:care_continuum>
```

Deletion

```
cqlsh:care_continuum> DELETE FROM clinical_events_by_patient_month
                  ... WHERE patient_id = 'DEMO-CRUD'
                  ... AND event_month = '2026-09'
                  ... AND event_time = '2026-09-01T00:00:00Z'
                  ... AND event_id = 'crud:1';
cqlsh:care_continuum>
```

Verifying the deletion

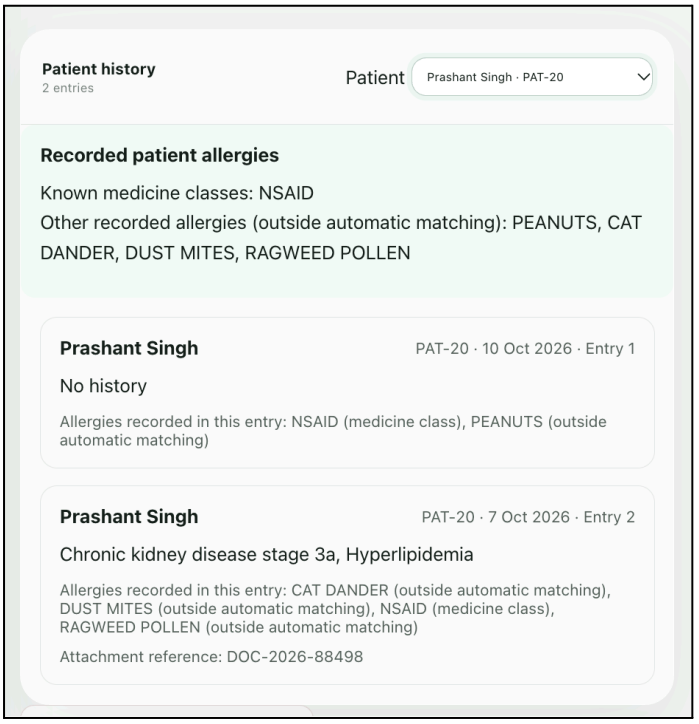
```
cqlsh:care_continuum> SELECT patient_id, event_month, event_time, event_id, event_type, summary
                  ... FROM clinical_events_by_patient_month
                  ... WHERE patient_id = 'DEMO-CRUD'
                  ... AND event_month = '2026-09';

 patient_id | event_month | event_time | event_id | event_type | summary
------------+-------------+------------+----------+------------+---------

(0 rows)
cqlsh:care_continuum>
```

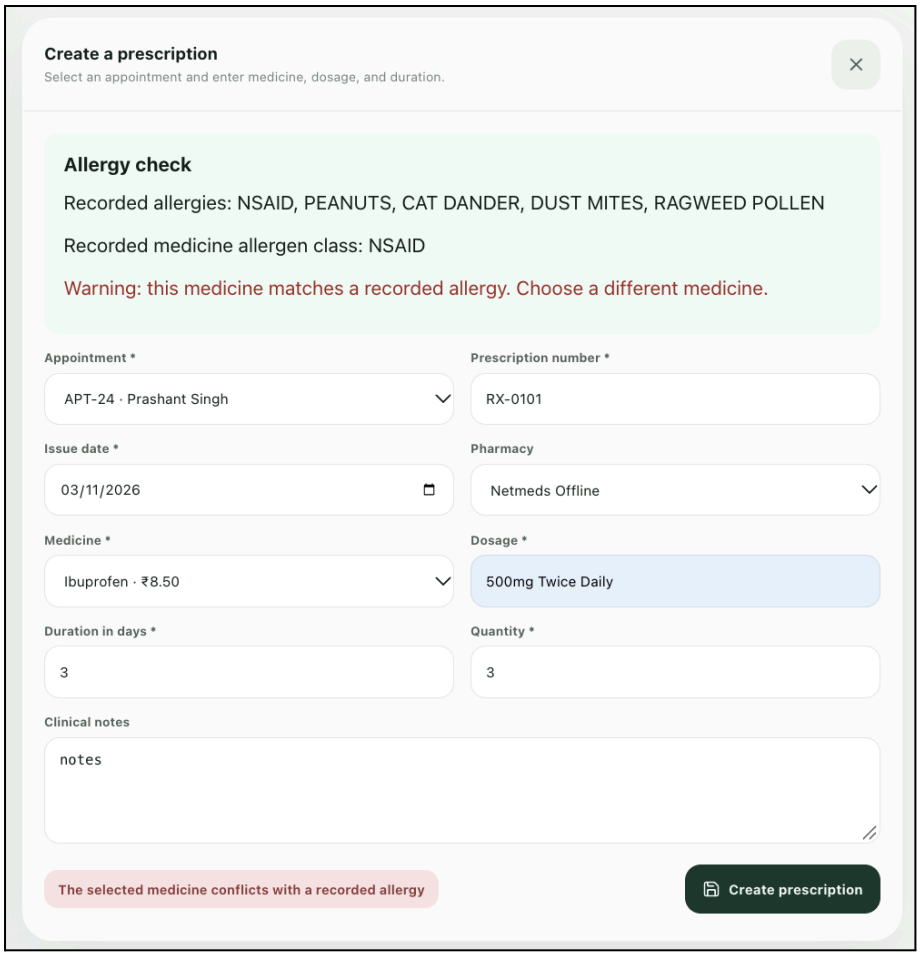
RESULT: Deletion executes successfully

TEST: Testing Allergic Prescription Trigger



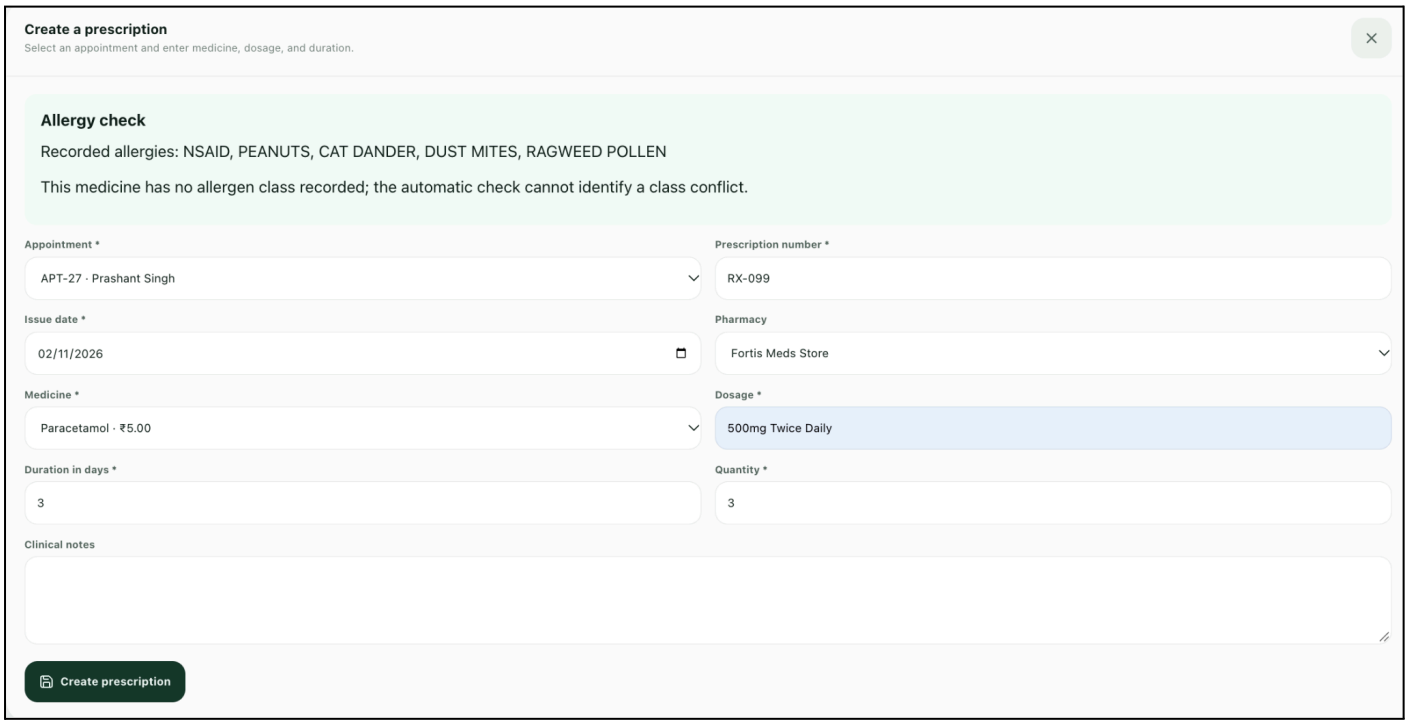
NSAID → includes Ibuprofen

If we try to prescribe Ibuprofen:



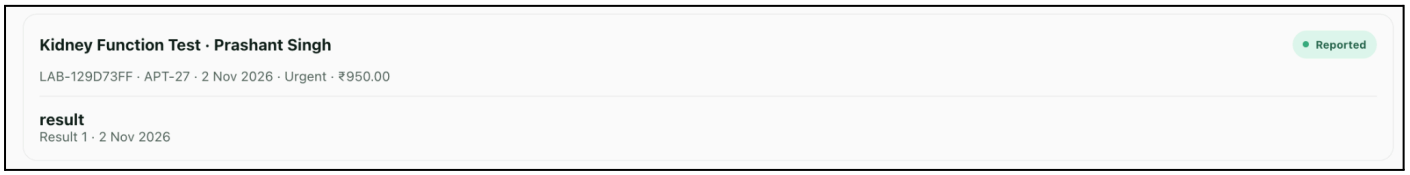
RESULT: Trigger works successfully

TEST: Testing invoice calculation procedure



Expected bill: 5 * 3 = 15

Then we add a lab order

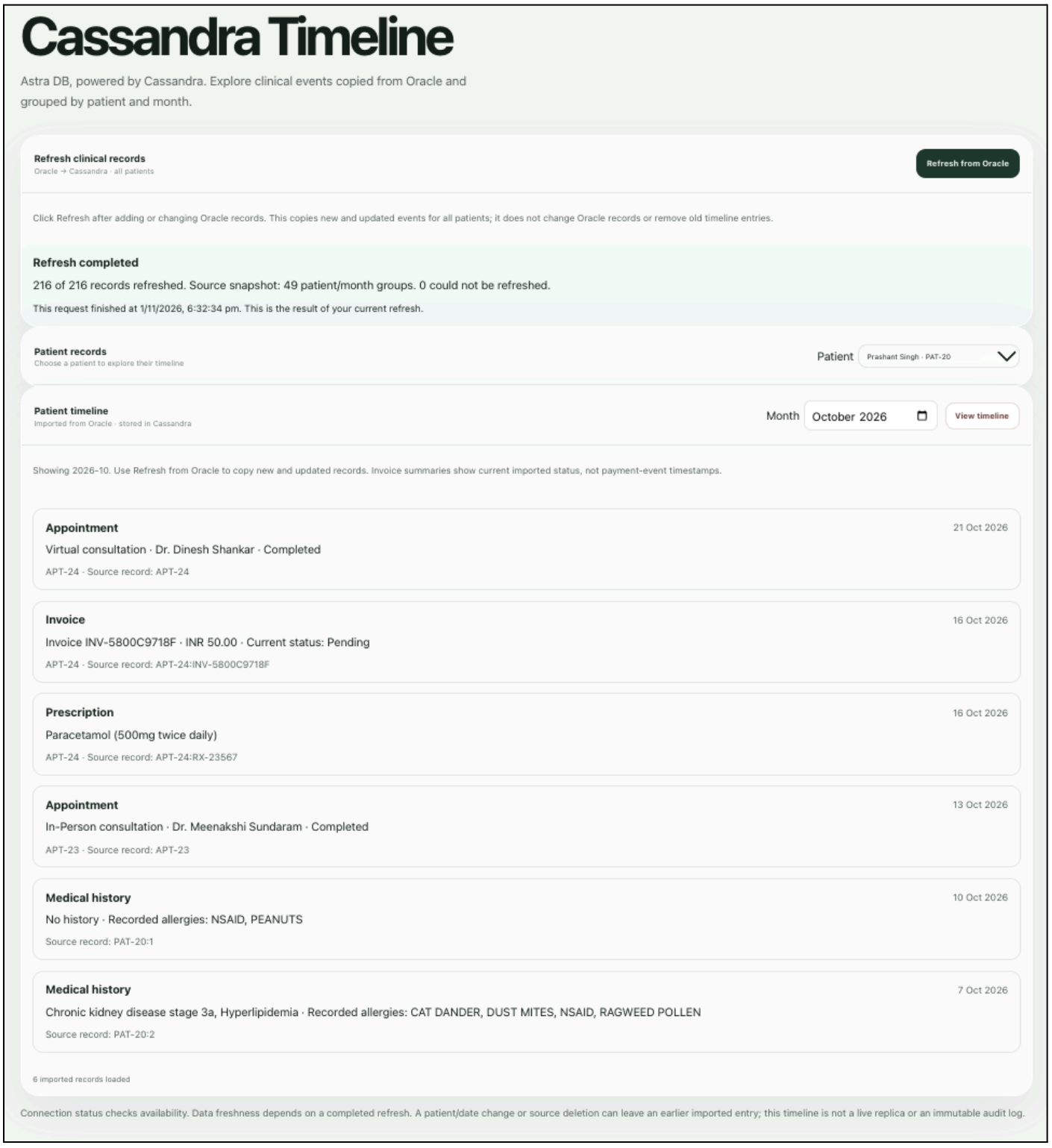


Expected bill: 15 + 950 = 965



RESULT: Invoice calculation works perfectly

TEST: Data retrieval from Cassandra



RESULT: Data is successfully retrieved from Cassandra and displayed in chronological order according to patient ID and month of event occurrence

14 Limitations

1. If a virtual appointment is booked, the website does not support direct video or audio calling. Instead, a meeting URL can be stored in the database which can be accessed.
2. Documents are not directly stored in the database. The links to their location in a cloud storage (like Google Drive) are allowed to be stored instead.
3. During invoice calculation, consultation fee is not added.
4. Transfer of records from Oracle to Apache is not automated. Manual refresh has to be done every time to retrieve the medical timeline.
5. The current refresh mechanism does not delete the outdated timeline entries in Cassandra.
6. The project is currently implementing a single node Cassandra environment for the academic prototype. Hence, the scaling is limited for this prototype.

15 Future Enhancements

1. Integration of real-time video consultation
2. Automated database synchronization with deletion and update reconciliation
3. Addition of consultation fee in invoice calculation
4. Multi-node Cassandra deployment to support scalability
5. Role-based access control with security enhancements like MFA, detailed audit logs and encryption policies.

16 Conclusion

This project presents a robust, scalable architecture for a Telemedicine Consultation and Electronic Health Records System. It integrates various clinical workflows into one logical unit. It stores operational data into the normalised relational database (Oracle), while storing the medical history as a chronological timeline.

The current architecture does not implement the total transfer of medical history data, but instead it just maps the records from the Oracle to events in Cassandra for an organised view of the timeline.

This unified system eliminates data fragmentation, increases accessibility and provides a sustainable infrastructure.

17 References

[1] Oracle Corporation, “Oracle Database PL/SQL Language Reference,” *Oracle Database Documentation*. [Online]. Available: https://docs.oracle.com/en/database/oracle/oracle-database/19/lnpls/. [Accessed: Nov. 1, 2026].

[2] Apache Software Foundation, “Apache Cassandra Documentation, Version 5.0.” [Online]. Available: https://cassandra.apache.org/doc/latest/. [Accessed: Nov. 1, 2026].

[3] Apache Software Foundation, “Data Modeling,” *Apache Cassandra Documentation*. [Online]. Available: https://cassandra.apache.org/doc/stable/cassandra/developing/data-modeling/index.html. [Accessed: Nov. 1, 2026].

[4] Apache Software Foundation, “The Cassandra Query Language (CQL),” *Apache Cassandra Documentation*. [Online]. Available: https://cassandra.apache.org/doc/latest/cassandra/developing/cql/index.html. [Accessed: Nov. 1, 2026].

[5] Oracle Corporation, “node-oracledb Documentation.” [Online]. Available: https://node-oracledb.readthedocs.io/en/latest/. [Accessed: Nov. 1, 2026].

[6] DataStax, “DataStax Documentation,” *DataStax Docs*. [Online]. Available: https://docs.datastax.com/en/home/documentation-list.html. [Accessed: Nov. 1, 2026].

[7] Meta Platforms, Inc., “React Reference Overview,” *React Documentation*. [Online]. Available: https://react.dev/reference/react. [Accessed: Nov. 1, 2026].

[8] OpenJS Foundation, “Node.js Documentation.” [Online]. Available: https://nodejs.org/en/docs. [Accessed: Nov. 1, 2026].

[9] Express.js, “Express — Node.js Web Application Framework.” [Online]. Available: https://expressjs.com/. [Accessed: Nov. 1, 2026].

[10] DataStax, “DataStax Node.js Driver for Apache Cassandra.” [Online]. Available: https://docs.datastax.com/en/developer/nodejs-driver/. [Accessed: Nov. 1, 2026].